Click-to-Source 3D

Implementation Plan — Auto-Tagging and Mesh Inspection

From Verified Research to Shipped Capability

---

1. Where This Starts

Six experiments established what automatic provenance capture can and cannot do. Three components emerged, and they stack rather than compete:

Component	Mechanism	Status
Location — file, function, line	Build-time `__source` injection	Not built; has precedent, no experiment needed
Routing — which mesh, which instance	`setMatrixAt` interception (H4)	Proven, 1205/1205 live
Args — what values produced it	Clone-chain join (H6)	Proven, 400/400 and 805/805 live

This document covers what gets built from here, in what order, and what mesh information the panel can surface at each point.

Two decisions frame it. **The target is the full composition**, not a partial tier — location plus routing plus args, which is the capability originally described as the goal. And **manual tagging remains as an override**: the resolver checks auto-generated provenance first and falls back to a hand-written `userData.sourceRef`, so existing tagged objects keep working, incorrect inferences stay fixable, and code the automation cannot reach still has a path.

---

2. The Method Being Implemented

2.1 Location

A Babel/Vite transform stamps every JSX element with `{fileName, functionName, line, column}` at build time, normalized onto `userData`.

Deliberately at build time rather than by reading React's fiber at runtime: React 19 removed `_debugSource` from fiber nodes, which broke this exact mechanism for several existing tools. Build-time stamping is version-independent.

This alone eliminates the hand-maintained `line:` integers — the defect class that produced three separate bugs in Stage 4 (a tag pointing at a comment, 176 versus 178, and 82 versus 6).

2.2 Routing

Patch `InstancedMesh.prototype.setMatrixAt`. The receiver is the destination mesh and the first argument is the destination slot — both are the call's own data, nothing is derived.

Two constraints from the experiments:

Constructor traffic must be filtered. Three's `InstancedMesh` constructor calls `setMatrixAt(i, _identity)` for every slot, so a prototype patch sees double traffic. The discriminator is object identity of the matrix argument: the constructor passes one shared singleton, application writes pass distinct clones.

The probe must install before first commit. Writes are once-only with no replay. If the patch is not live when the scene mounts, it captures nothing.

2.3 Args — the clone chain

Three interception points joined by object identity:

```
updateMatrix() on the dummy   ->  transform state at generation time
       |
Matrix4.prototype.clone()     ->  distinct object identity per instance
       |
setMatrixAt(index, matrix)    ->  (mesh, instanceId)
```

`dummy.matrix` is the same object every iteration, so only the clone creates the identity the join needs.

The pairing filters by receiver identity, not by count. Injecting one foreign `Matrix4.clone()` per instance broke the counting rule outright while receiver-filtered pairing held at 200/200. The clone ratio is 2:1 in Trees and 1:1 in GroundCover, so any count-based rule is codebase-specific and would ship a silent failure.

A count-aware sweep is required. If an instance count shrinks on a reused mesh, stale slots still render and their records are indistinguishable by `(mesh, instanceId)` alone.

Measured cost on a real page load: 2.12 ms full record at mount, 2410 clone calls all originating from the application, zero from three.js, drei, R3F, or OrbitControls.

2.4 Scope boundary

Under library-owned instancing — drei `<Instances>` writes transforms directly into a `Float32Array` — there is no `setMatrixAt` call anywhere in the program. The join has no destination side.

This is an absence, not a configuration choice. The implementation should detect it (instances rendered, zero intercepted writes) and say so, rather than silently mis-tagging. Two claims must stay separate in any documentation: **hand-rolled instancing is solved; library-owned instancing is not.**

---

3. Mesh Attributes — Adding Now

Everything in this section is a read off the live object at click time. No capture, no plugin, no build tooling.

The unlock is that `selectedObject` and `instanceId` are **already in the overlay store**; `GenerationTrace` reads neither and renders only `sourceRef`.

These attributes also cannot drift. Anything read fresh at click time is correct by construction — which directly addresses the failure mode behind three Stage 4 bugs.

3.1 Instance identity

Field	Source
Instance N of M	`instanceId` (already stored) + `mesh.count`
Object type	`object.type` — `Mesh` vs `InstancedMesh`
Mesh uuid	`object.uuid`
Per-instance colour	`mesh.getColorAt(instanceId, c)` where `instanceColor` exists

3.2 Geometry

Field	Source
Vertex count	`geometry.attributes.position.count`
Triangle count	`index ? index.count / 3 : position.count / 3`
Indexed or non-indexed	`geometry.index !== null`
Attributes present	`Object.keys(geometry.attributes)`
Bounding box / sphere	`geometry.boundingBox`, `geometry.boundingSphere`
Geometry memory	Sum of attribute `array.byteLength`
Shared geometry	Compare `geometry.uuid` across meshes

Two of these have already cost real debugging time. `boundingSphere: null` **is the diagnostic that would have collapsed the Trees raycast investigation into a glance** — InstancedMesh raycasts fell through to Terrain for want of `computeBoundingSphere()`. And **shared-geometry detection surfaces the Trees foliage trap directly**: those meshes share one geometry and separate only by material, while GroundCover's variants share one material and separate only by geometry. An implementation keying on one alone works on one component and silently fails on the other.

Indexed-versus-non-indexed is also directly relevant post-normals-fix, where removing a vestigial `toNonIndexed()` took the buffer from 345,600 to 58,081 vertices.

3.3 Material and render state

Field	Source
Material type	`material.type`
Colour	`material.color.getHexString()`

Transparency flags	`transparent`, `opacity`, `depthWrite`, `depthTest`, `side`, `blending`
Render order	`object.renderOrder`
Frustum culled	`object.frustumCulled`
Live shader uniforms	`material.uniforms`
Shared material	Compare `material.uuid`

Water carries the exact flag combination that produces classic transparency bugs — `transparent`, `depthWrite: false`, `side: 0`, `renderOrder: 2`. Seeing them together turns a bisect into a glance, and `frustumCulled: false` explains why it never disappears at oblique angles. Live uniforms are also a free liveness check: `uTime` ticking means the material is genuinely animating.

3.4 Transform — including a notable one

Field	Source
Local transform	`object.position`, `rotation`, `scale`
World transform	`object.getWorldPosition(v)`, etc.
Per-instance transform	`mesh.getMatrixAt(instanceId, m)` **then decompose**

The last row deserves attention. Decomposing an instance matrix yields position, scale, and rotation — which for Trees is exactly `x`, `z`, `height`, `scale`, `yaw`. **The current `args` payload is recoverable for free at click time**, from the GPU-bound buffer rather than a generation-time recording.

This does not make the clone chain redundant: H6 supplies generation-time values and the loop-iteration link that Tier 2's intermediates depend on. But for pure transform values, no capture is needed at all. And if the two ever disagree, something mutated the instance after generation — which is itself diagnostic.

World-versus-local also matters more than it appears. Stored `args` hold the *authored* x/z, which agree with reality only because no parent group is currently transformed. The moment one is, they diverge silently.

3.5 Hierarchy and liveness

Field	Source
Parent chain	Walk `object.parent`
Still in scene	Walk to root

Liveness directly addresses H6's stale-mesh hazard: after HMR, orphaned meshes remain reachable from held references while no longer rendering.

3.6 Source snippet

The `/__cts/read-file` endpoint already exists. Displaying the actual lines around `sourceRef.line` requires no new mechanism — and is arguably closer to the phrase "click to source" than anything else currently on screen.

3.7 Recommended first three

Instance identity, bounding volumes, and the source snippet. All small, all read-only by construction, and each has already cost debugging time or directly answers the tool's own name.

---

4. Mesh Attributes — Future Versions

Each of these needs capture, analysis, or infrastructure that does not exist yet.

4.1 Generation context

Field	Requires
Seed	Capture at generator entry. Does not vary per instance, so needs no routing — attach once per generation pass
Generator constants (`terrainSize`, `count`)	Same mechanism as seed
RNG draw index	A counting RNG wrapper in the consumer. Would allow replaying generation to the exact call that produced an instance

Seed is the cheapest genuinely new capability here and closes most of what the current args payload lacks.

4.2 Placement predicates

Field	Requires
`suitability`, `slope`, `altNorm`, `maxDrop`	H2-style call capture, clock-joined to the clone chain
Which gate rejected a candidate	The same, extended to rejection paths

This is the highest-value future item and the one with a clear path. H2 captured all of these at 200/200 but could not route them for want of a join key — which H6 now supplies. Values computed in loop iteration N would attach to whichever clone emerged from iteration N.

Near-misses are the interesting case: `altitudeFade: 0.03` means a tuft barely survived the fade, which is a completely different debugging story from `0.98`. Procedural bugs are usually predicate bugs, not value bugs.

Untested. It is the natural seventh experiment and smaller than H6 was — the plumbing exists; the question is only whether clock-stamping holds when capture and `updateMatrix` sit in the same iteration.

4.3 Value provenance and blast radius

Field	Requires
Is this arg a literal, prop, or shared config	A static AST resolve pass
What else changes if I edit this	Cross-file analysis

This generalises the `terrainSize` incident, where a field appeared editable but had no literal to write to. `ALPINE_TERRAIN.worldSize` feeds Terrain, Trees, GroundCover, and Water — editing it from any one panel

has scene-wide effect and nothing surfaces that.

Related, and already logged as a TODO: validating args at panel-open time so unresolvable fields render read-only rather than failing on save.

4.4 GLSL and shader provenance

Explicitly out of scope for v0.1 and deferred to a future version.

Field	Requires
Shader uniform provenance — which source line set this uniform	Tracing JS-side uniform assignment
Values inlined in GLSL source	GLSL parsing; the Babel editor cannot reach into shader strings
TSL / node-material graphs	A different representation entirely
Which shader chunk produced a fragment result	Substantially deeper analysis

The boundary is concrete in this project. Water's `uDeepColor` and `uShallowColor` are editable **only because** the JS constants were lifted out and passed in as uniform inputs — a workaround that happens to hold here, not a general property. The wave frequencies and amplitudes are hardcoded inside the vertex shader's template literal and are invisible to every mechanism tested.

Two distinct problems hide under "shader support":

Uniforms set from JS are tractable — the assignment is ordinary JavaScript and could be traced with existing techniques. This is the realistic first step.

Values inlined in GLSL need a GLSL parser and a patcher that round-trips shader source safely. That is a materially different engineering problem than the current Babel and magic-string pipeline, and building it should be treated as its own project rather than an extension.

Worth stating plainly in documentation: a shader-heavy project gets location and material state from this tool, and nothing about the values inside its shaders.

4.5 Other future items

Field	Requires
Which generation pass created this	Explicitly staged generation, which does not exist yet
Git blame for the source line	Git integration
Construction call stack	Capture at construction time
Draw-call cost attribution	Renderer-level instrumentation

---

5. Sequencing

5.1 Order

First — the packaging blocker. Barrel files use extensionless re-exports, which Node's ESM resolver rejects with `ERR_MODULE_NOT_FOUND`. It works today only because Vite bundles it; it breaks the moment anyone imports the published package directly. Small fix, blocks publishing entirely.

Second — the free mesh attributes (Section 3). Independently shippable, no dependency on any of the auto-tagging work, and each item is a read off an object already in the store.

Third — location stamping. Eliminates manual `line:` maintenance and is a prerequisite for the rest regardless of what else gets built.

Fourth — a checkpoint. Tag a release with everything working and nothing blocking. This is what makes the remaining work safe to attempt.

Fifth — routing and the clone chain. The research is finished; what remains is engineering rather than discovery.

5.2 Why this order

Every step before the checkpoint is independently useful and independently shippable. If the composition work stalls, the fallback is a working, published tool rather than a half-migrated one.

The counter-argument is real: the composition is the differentiator and it is already proven, so delaying it costs perhaps a week. The judgement here is that a week is cheap insurance for having something shippable at every point — but it is a timeline call, not a technical one.

5.3 Effect on later stages

Stage 5 — unaffected mechanically, but the README's claim changes with what ships. The barrel-export fix belongs here regardless.

Stage 6 (MCP) — improved. A prior review found `search_by_generator` assumed a queryable index that nothing builds, and `search_by_seed` needs a seed captured nowhere. The join produces exactly that index: records keyed by `(mesh, instanceId)` with location attached, making `search_by_generator` a filter. `search_by_seed` still needs seed added to the captured set — a tagging decision, not a mechanism gap.

Stage 6.5 — building the composition *is* Stage 6.5. It was scheduled as optional and post-launch; doing it now moves it forward and effectively closes it. That is a real roadmap change and should be recorded rather than allowed to happen implicitly.

Stage 7 — the scope boundary must be stated: hand-rolled instancing supported, library-owned instancing not, and the package patches Three.js prototypes globally.

---

6. Constraints Carried Forward

Five findings from the experiments that any implementation must respect:

Filter by receiver identity, not by count. The counting rule passes on this codebase and breaks on any code cloning a matrix mid-loop.

Install the probe before first commit. Writes are once-only with no replay.

Sweep on count change. Shrinking instance counts leave stale slots that look live.

Key on both material and geometry. Trees separates by material, GroundCover by geometry. Either alone silently fails on one of them.

Detect and refuse the unsupported case. Instances rendered with zero intercepted writes means library-owned instancing. Warn rather than mis-tag — three separate mechanisms produced results in testing that looked correct and were wrong, and each would have passed a demo.